

load-parser

Fedi Khala

June 2026

1 Architecture du Loader Factory et Parser Factory

L'objectif principal est de construire une architecture extensible permettant de supporter plusieurs types de sources de données tout en fournissant un format interne unifié pour les étapes suivantes du pipeline RAG.

1.1 Principe Général

L'architecture est divisée en plusieurs couches :

```
Source
  ↓
Loader
  ↓
RawDocument
  ↓
Parser
  ↓
ParsedDocument
  ↓
Chunker
  ↓
Chunks
```

Loader :

- Récupère les données depuis une source.
- Ne réalise aucun traitement métier.
- Ne fait ni chunking ni embedding.

Parser :

- Analyse le contenu brut.
- Extrait la structure sémantique.
- Produit un document normalisé.

1.2 Modèle Unifié de Document

Tous les types de sources sont convertis vers un format commun :

```
class ParsedDocument:
    id: str
    source_type: str
    metadata: dict
    sections: list[Section]
    tables: list[Table]
    images: list[Image]
```

Ce modèle permet de traiter de manière identique :

- PDF
- DOCX
- TXT
- Markdown
- CSV
- Excel
- Bases de données
- Sites Web

2 Loader Factory

Le Loader Factory sélectionne automatiquement le chargeur adapté à la source.

```
loader = LoaderFactory.get_loader(source)
raw_document = loader.load()
```

2.1 Documents

2.1.1 PDF Loader

Responsabilités :

- Lecture du fichier PDF.
- Extraction du contenu brut.
- Retour d'un objet RawDocument.

2.1.2 DOCX Loader

Responsabilités :

- Lecture du document Word.
- Extraction du contenu brut.

2.1.3 TXT Loader

Responsabilités :

- Lecture des fichiers texte.

2.1.4 Markdown Loader

Responsabilités :

- Lecture des fichiers Markdown.

2.2 Données Tabulaires

2.2.1 CSV Loader

Sortie :

```
RawDocument(dataframe)
```

2.2.2 Excel Loader

Sortie :

```
{  
    sheet1: dataframe,  
    sheet2: dataframe  
}
```

2.2.3 Google Sheets Loader

Fonctionnalités :

- Connexion via Google Sheets API.
- Récupération des feuilles sous forme de DataFrame.

2.3 Bases de Données

2.3.1 PostgreSQL Loader

Configuration :

```
{  
  "host": "...",  
  "database": "...",  
  "query": "SELECT * FROM employees"  
}
```

Sortie :

DataFrame

2.3.2 MySQL Loader

Connexion à une base MySQL et récupération des données.

2.3.3 SQL Server Loader

Connexion à Microsoft SQL Server.

2.3.4 MongoDB Loader

Sortie :

JSON Documents

2.4 Sources Web

2.4.1 URL Loader

Entrée :

<https://docs.company.com>

Sortie :

HTML Content

2.4.2 Website Crawler Loader

Fonctionnalités :

- Exploration automatique du site.
- Collecte de plusieurs pages.

2.4.3 Sitemap Loader

Fonctionnalités :

- Lecture du fichier sitemap.xml.
- Extraction automatique des URLs.

3 Parser Factory

Le Parser Factory transforme un RawDocument en ParsedDocument.

```
parser = ParserFactory.get_parser(raw_document)
parsed_document = parser.parse()
```

3.1 PDF Parser

Extraction de :

- Titres
- Paragraphes
- Tableaux
- Images

Exemple :

```
Employee Benefits
  Paragraphs
  Tables
  Images
```

3.2 DOCX Parser

Extraction de :

- Heading 1
- Heading 2
- Paragraphes
- Listes
- Tableaux

3.3 Markdown Parser

Transformation des titres Markdown :

```
# Title
## Subtitle
### Section
```

en sections structurées.

3.4 CSV Parser

Conservation de la structure tabulaire :

```
Table(
  headers=[],
  rows=[]
)
```

3.5 Excel Parser

Pour chaque feuille :

- Nom de la feuille
- Colonnes
- Types de données
- Lignes

3.6 Database Parser

Transformation des résultats SQL en objets Table.

Exemple :

```
Table(
  name="employees",
  columns=[]
)
```

Métadonnées :

```
{
  table_name: "employees",
  database: "postgres"
}
```

3.7 HTML Parser

Extraction à l'aide de BeautifulSoup :

- Titre
- H1, H2, H3
- Paragraphes
- Tableaux
- Blocs de code

4 Organisation du Projet

src/

loaders/

documents/

pdf_loader.py
docx_loader.py
txt_loader.py
markdown_loader.py

tabular/

csv_loader.py
excel_loader.py
google_sheets_loader.py

database/

postgres_loader.py
mysql_loader.py
mongodb_loader.py
sqlserver_loader.py

web/

url_loader.py
crawler_loader.py
sitemap_loader.py

parsers/

pdf_parser.py
docx_parser.py
markdown_parser.py

```
csv_parser.py  
excel_parser.py  
database_parser.py  
html_parser.py
```

5 Objectif Final

- Loader Factory : *Bring data from anywhere.*
- Parser Factory : *Convert everything into one unified document schema.*
- Chunking Factory : applique ensuite la meilleure stratégie de découpage indépendamment de la source d'origine.

Cette architecture permet de supporter les domaines RH, Finance, IT, Juridique ou tout autre type de documents avec une seule pipeline RAG unifiée.